# Poker Bot Using RL

Mark Calvelo
*Bellini College of Artificial Intelligence and Computing*
*University of South Florida*
Tampa, Florida, USA
calvelo@usf.edu

Anthony Martini
*Bellini College of Artificial Intelligence and Computing*
*University of South Florida*
Tampa, Florida, USA
acmartini@usf.edu

***Abstract*** **report outlines the development of a Reinforcement Learning agent for multiplayer No-Limit Texas Hold To navigate imperfect information and maximize long-term expected value, the system utilizes a dual-base model approach: an Equity Model for mathematical winning probabilities and an Imitation Model pre-trained on human play to provide a strategic baseline. These base models are used to train an advanced LSTM network that evaluates sequential game states and history. While the LSTM demonstrated strong profitability against the base models, the project concludes that exploring Counterfactual Regret Minimization (CFR) is ultimately necessary to achieve an unexploitable strategy in competitive play.**

## I. Introduction

Developing a successful AI for poker requires navigating a complex environment characterized by imperfect information, necessitating optimal decision-making even when crucial data is hidden. The primary objective of this project is the development of a winning AI bot for multiplayer No-Limit Texas Hold Rather than focusing strictly on winning individual hands, the core objective is to maximize long-term expected value. To achieve this, the bot must dynamically adapt, recalculating odds and shifting strategies across all phases of the game from the pre-flop to the river. Furthermore, the system is designed to emulate natural, unpredictable human decision-making to create a realistic opponent.

## II. Background

The poker environment consists of the rules, the deck of cards, and the other players at the table. The state space represents the specific situation the bot faces, which includes its hole cards, the community cards, the pot size, and the betting history of its opponents. During its turn, the action space allows the bot to make distinct choices, primarily folding, calling, or raising specific amounts. The reward signal driving the learning process is defined as the net change in the chip stack at the conclusion of a hand. Through the learning process, the bot adjusts its strategy over millions of simulated hands, favoring actions that yield positive chip rewards and discarding actions that lead to losses.

## III. Methodology

The decision process evolved from standard architectures into a parallel training environment, utilizing two separate base models to train a final, advanced agent.

### A. Initial Architecture

The initial approach utilized Self-Play alongside Actor-Critic and Proximal Policy Optimization algorithms. This was built on a simple 3-layer neural network with ReLU activation. However, these early models suffered from naive aggression, exhibiting predictable or biases. Furthermore, they demonstrated a strategic failure to distinguish between high-equity hands and bluffs. To solve this, hand equity was incorporated as a core input.

### B. Hand Equity Base Model

The first base model mathematically calculates the probability of winning. It processes 139 inputs, including player hole cards, board cards, hand strength, and rank/suit counts. The architecture passes this data through four dense blocks (139 1024, 1024 1024, 1024 512, and 512 256). The model utilizes Batch Normalization and dropout, culminating in a final output layer (256 1) that yields a single equity percentage.

### C. Imitation Base Model

After working on the first two approaches, we wanted to find another way to instill human like strategy. Upon further research we found a poker dataset from the University of Computer Poker Research group. The dataset contained over 10,000,000 games of poker collected in 2001 [3]. Our approach was to fine-tune the backbone of the model on the dataset and then use the standard Actor-Critic with PPO to fine tune the model.

*1) Imitation Dataset:* The dataset upon extraction contained different archives for different modes of Texas Hold It ranged from tournament games to normal games. For this purpose, we only extracted normal play games. We then created 3 different datasets after doing separate training experiments:

Whole Dataset
Dataset filtered by hand equity
Dataset filtered by player with percentage

Our initial data set consisted of all the games we extracted from the normal play games. In parsing the dataset, we find that it is structured as a relational database. We observe 3 main files/directories when examining the data:

pdb
hdb

- hroster

Upon looking at the data, hroster provides information for the *individual games* containing information of which players are at the table and their position, each player in hroster can then be mapped to pdb which contains individual entries for each player mapped by name and game ID. For each player in pdb, it contains the information for the player in each game they played such as the action for pre-flop, flop, turn, river, and showdown. Each game in a player's pdb entry contains a game ID that we then use to map to hdb which contains information for the pot and how much each player bet throughout the stages of the game. We were then able to take this information and format it into tensors we can insert into the model for quick training. The limitations we noticed with the dataset is that the player's hand is not revealed unless they continue to showdown which is the last phase of the game. If they flop beforehand, the player's cards is not shown. This made it difficult to determine whether they folded due to low hand equity or if they made a bad decision. The solution was to just take the hands that go to showdown as we need the hand information to teach the bot how to approach the game.

*2) Dataset Pretraining:* The initial pre-training with the whole dataset yielded poor results where the model was unable to play against previous iterations of the bot with strategy. We theorized that this is due to two things: the bot was learning off of players who do not play optimally and have a low win percentage and games where players are playing with a hand with low equity. We then proceeded to recompile the dataset and extended the model inputs to include hand equity into the tensors (from 119 to 120). We compute the equity for the hand using phevaluator [1] and discard the hand if the equity is lower than 0.8 which should give us hands where the hand has a high probability of winning the pot and only take hands from players with a high overall win percentage. This gives us players who tend to play with strategy and win the majority of their games. Pre-training with this dataset then conducting reinforcement learning by playing against previous bots yielded good results displayed in Table II. This gave us a good baseline to further train a more advanced bot with this bot that plays with human strategy.

### *D. Advanced LSTM Model*

The final advanced architecture was trained using the pre-computed Equity and Imitation bots. The LSTM takes 166 inputs, combining the game state and history with hole cards, board cards, rank/suit counts, hand strength, and the simple Equity Neural Network output. It features two Encoder Blocks ($166 \rightarrow 512$ and $512 \rightarrow 128$) and utilizes Layer Normalization. An LSTM core ($128 \rightarrow 128$) serves as the Recurrent Memory State. Output heads handle the Actor actions ($128 \rightarrow 3$ for Fold, Call, Raise), Actor bet sizing ($128 \rightarrow 2$ for Beta Distribution Parameters), and the Critic value ($128 \rightarrow 1$).

## IV. Experimental Setup

The Equity Model was trained on a dataset of 2 million hands for each stage of the game. Labels were generated using 2,500 Monte Carlo simulations per hand phevaluator tool [1].For optimization on the Flop, all 169 unique hands were pre-calculated with 25,000 simulations each and stored as a JSON file. For the River stage, the exact 1,081 remaining opponent combinations were simulated, eliminating the need for Monte Carlo approximations.

For the LSTM training phase, games were sampled from a precomputed 100-million hand dataset. The opponent was chosen randomly between the equity bot and the imitation bot. Weights were updated every 10,000 hands, completing 100 weight updates over 1 million hands.

## V. Results

The Equity Model was tested on 10,000 hands per stage, achieving an accuracy error rate of less than 0.29% across all stages, while also maintaining a 0.9999 correlation . These metrics are summarized in Table I.

TABLE I
EQUITY MODEL PERFORMANCE METRICS ACROSS GAME STAGES

| Stage | MSE | MAE (%) | Correlation |
|---|---|---|---|
| Flop | 0.00001099 | 0.26 | 0.9999 |
| Turn | 0.00001259 | 0.28 | 0.9999 |
| River | 0.00002181 | 0.20 | 0.9999 |

Table II demonstrates the overall performance of the Imitation Model against previous bots. After 10,000 games, the Imitation Model Hero secured 1728.48 wins which is the combined result of being the only winner and splitting the pot with the other winners in case of a N-player tie throughout the 10,000 games. The win distribution against the five villains is detailed in Table II. The Total Hero Reward was $407,190.90, resulting in an Average Hero Reward of $40.72 per game against the villains.

TABLE II
IMITATION MODEL WIN DISTRIBUTION (10,000 GAMES)

| Agent | Wins |
|---|---|
| Hero | 1728.48 |
| Villain 1 | 1702.12 |
| Villain 2 | 1656.17 |
| Villain 3 | 1643.55 |
| Villain 4 | 1656.28 |
| Villain 5 | 1728.48 |

During testing with 1,000 chip starting amounts, the final LSTM model performed strongly against both base models. Over 200,000 hands, it averaged 85.0 chips per hand against the equity model and 78.4 chips per hand against the imitation model.

## VI. Discussion and Challenges

The project successfully built a strong model capable of playing with a mix of calculated and human-like strategy. The incorporation of distinct state inputs and base models addressed the strategic failures of earlier standard architectures.

However, significant limitations remain. Standard RL architectures can be exploited in games characterized by imperfect, hidden information. While the LSTM model performed well against its base benchmarks, the models underperformed in competitive league play. This suggests that a larger model capacity or a transition to alternative strategic frameworks is required for true competitive viability.

## VII. Conclusion and Future Work

This research demonstrates the efficacy of combining mathematical equity features with imitation learning to train a recurrent reinforcement learning agent for No-Limit Texas Hold 'em. While the resulting model exhibits strong profitability against its baseline components, its vulnerability to exploitation highlights the limits of standard RL in hidden-information domains. Future work will focus on exploring Counterfactual Regret Minimization (CFR). CFR is utilized by other top poker tools [2] because it is significantly more effective at reaching a Nash Equilibrium and achieving an unexploitable, game-theory-optimal strategy.

## References


[1] H. R. Lee, "PokerHandEvaluator: An efficient poker hand evaluation algorithm and its implementation," GitHub repository, https://github.com/HenryRLee/PokerHandEvaluator, 2026.

[2] Brown, Noam, and Tuomas Sandholm, "Superhuman AI for Multiplayer Poker." *Science*, vol. 365, no. 6456, 30 Aug. 2019, pp. 885–890. Science, https://doi.org/10.1126/science.aay2400.

[3] Computer Poker Research Group, *IRC Poker Database*. University of Alberta, https://poker.cs.ualberta.ca/irc_poker_database.html.